Gig Cashflow Index (GCI) – Autonomous MVP Build Specification
Agent Execution Playbook

MISSION
Build a complete working MVP of the Gig Cashflow Index (GCI) system.
Purpose: Evaluate creditworthiness of gig workers using financial transaction behavior.

SYSTEM CONSTRAINTS
Backend: Python + FastAPI
Machine Learning: scikit■learn, pandas, numpy
Database: PostgreSQL + SQLAlchemy
Frontend: React + Vite
Deployment: Docker + Docker Compose
Visualization: Graphviz + Matplotlib

DATASET REQUIREMENTS
borrowers: 500
transactions per borrower: 300–600
total transactions: ~200000

Dataset schema:
transaction_id
borrower_id
timestamp
amount
direction
counterparty_id
counterparty_type
payment_mode
category
description
balance_after_txn

PROJECT DIRECTORY STRUCTURE

gci/
■ data/
■ ■ raw/
■ ■ synthetic/
■ ■ processed/
■
■ backend/
■ ■ api/
■ ■ models/
■ ■ services/
■ ■ database/
■
■ ml/
■ ■ training/
■ ■ features/
■ ■ anomaly/
■
■ frontend/
■ scripts/
■ docker/
■ docs/

DATASET ACQUISITION
Search and download public financial transaction datasets from Kaggle or GitHub.
Store raw datasets in:
data/raw

SYNTHETIC DATA GENERATION

Create:
scripts/generate_transactions.py

Generate synthetic gig worker financial behavior using:

faker
numpy
pandas

Output file:
data/synthetic/gig_transactions.csv

DATA CLEANING PIPELINE

Create:
scripts/clean_dataset.py

Steps:
1 remove null rows
2 normalize timestamps
3 convert amounts to float
4 encode direction
5 categorize transaction description

Output:
data/processed/transactions_clean.csv

FEATURE ENGINEERING

Module:
ml/features/feature_engineering.py

Features:
avg_monthly_income
income_std_dev
income_volatility
expense_income_ratio
weekly_income_variance
liquidity_buffer
recurring_expense_count
round_trip_transaction_count
dormancy_score

Output:
data/processed/features.csv

MANIPULATION DETECTION

Module:
ml/anomaly/manipulation_detector.py

Algorithm:
IsolationForest

Outputs:
manipulation_label
anomaly_score
penalty_multiplier

Penalty rules:
clean = 1.0
suspicious = 0.75
high_risk = 0.50

PROBABILITY OF DEFAULT MODEL

Module:
ml/training/train_pd_model.py

Model:
GradientBoostingClassifier

Training process:
train/test split
cross validation
feature importance extraction

Saved model:
models/pd_model.pkl

GIG CASHFLOW INDEX CALCULATION

Module:
ml/training/gci_score.py

Score range:
300–900

Formula:
GCI = base_score
      + income_stability_weight
      + liquidity_weight
      - volatility_penalty
      - anomaly_penalty

DATABASE IMPLEMENTATION

backend/database/models.py

Tables:
borrowers
transactions
features
gci_scores
manipulation_flags

FASTAPI BACKEND

backend/api/main.py

Endpoints:
POST /upload-transactions
GET /borrower/{id}
GET /gci/{id}
GET /anomaly/{id}
GET /model/feature-importance

Run:
uvicorn backend.api.main:app --reload

REACT FRONTEND

Initialize:
npm create vite@latest frontend
cd frontend
npm install

Pages:
Dashboard
Borrower view
GCI score view
Transaction chart
Anomaly alerts

DASHBOARD COMPONENTS

Display:
GCI Score
Probability of Default
Income Stability
Liquidity Score
Anomaly Risk
Transaction Charts

SYSTEM DIAGRAMS

docs/architecture.dot

Pipeline:
Borrower Data → Cleaning → Feature Engineering → Manipulation Detection → Risk Model → GCI Score

DOCKERIZATION

Create:
Dockerfile.backend
Dockerfile.frontend
docker-compose.yml

Run:
docker compose up --build

SYSTEM VALIDATION

Test cases:
income spike attack
round trip transactions
expense suppression
dormant account activation

API DOCUMENTATION

Access FastAPI documentation at:
http://localhost:8000/docs

FINAL OUTPUT

Repository must contain:
backend/
frontend/
ml/
data/
scripts/
docker/
docs/
README.md

System launch command:
docker compose up